

LeetCode 463: Island Perimeter

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 22, 2026

© 2026 Arpit Raj. All rights reserved.

Approach 1: Iterative Matrix Traversal

This approach iteratively scans the entire grid. Every time a land block (1) is encountered, it initially contributes 4 edges to the perimeter. Then, it looks at its immediate neighbors (top, bottom, left, right). If an adjacent cell is also land, they share an edge. We subtract 1 from the total perimeter for each shared edge found.

```
class Solution {
public:
    int islandPerimeter(vector<vector<int>>& grid) {
        int n = grid.size();
        int m = grid[0].size();
        int ans = 0;
        vector<pair<int,int>> dir = {{-1,0}, {0,-1}, {1,0}, {0,1}};

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(grid[i][j] == 1) {
                    ans += 4;
                    for(auto it : dir) {
                        int nr = i + it.first;
                        int nc = j + it.second;
                        if(nr >= 0 && nr < n && nc >= 0 && nc < m) {
                            if(grid[nr][nc] == 1) {
                                ans--;
                            }
                        }
                    }
                }
            }
        }
        return ans;
    }
};
```

Approach 2: Breadth-First Search (BFS)

This method employs Breadth-First Search (BFS) to traverse the island. A separate 2D seen array is maintained to ensure the original grid's integrity and avoid infinite loops. For each cell processed, we check its neighbors. The perimeter contribution of a single land cell is essentially $4 - (\text{number of adjacent land cells})$.

```
class Solution {
public:
    int islandPerimeter(vector<vector<int>>& grid) {
        int n = grid.size();
        int m = grid[0].size();
        int ans = 0;
        vector<pair<int,int>> dir{{-1,0}, {0,-1}, {1,0}, {0,1}};
        queue<pair<int,int>> q;

        // iss baar seperate seen 2d array lagega kyunki agar original array
        // me change krenge to number of neighbours can be affected
        vector<vector<int>> seen(n, vector<int>(m, 0));

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(grid[i][j] == 1 && !seen[i][j]) {
                    seen[i][j] = 1;
                    q.push({i,j});
                    while(!q.empty()) {
                        int r = q.front().first;
                        int c = q.front().second;

                        // har ek queue level ka size ek particular cell ka
                        // no. of neighbour hoga
                        // agar 4 neighbour to perimeter contri=0;
                        // agar 3 neighbour to perimeter contri=1;
                        // agar 2 neighbour to perimeter contri=2;
                        // agar 1 neighbour to perimeter contri=3;
                        // agar 0 neighbour to perimeter contri=4;
                        int neighbour = 0;

                        q.pop();
                        for(auto it : dir) {
                            int nr = r + it.first;
                            int nc = c + it.second;

                            if(nr >= 0 && nc >= 0 && nr < n && nc < m) {
                                if(grid[nr][nc] == 1) {
                                    neighbour++;

                                    if(!seen[nr][nc]) {
                                        seen[nr][nc] =
1; // seen sirf isliye he ki same cell baar baar queue me na jaaye
                                        q.push({nr,nc});
                                    }
                                }
                            }
                        }
                    }
                }
            }
        } // iss loop ke baad pata chal jaayega ki har 1
        marked cell ke kitne neighbour he
    }
};
```

```
        ans += 4 - neighbour;  
    }  
    }  
    }  
    return ans;  
};
```